SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Analytical Problem Solving

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

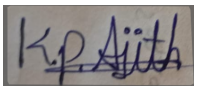
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I K.P. Ajith Kumar/192425139 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **Name of the student : K.P. Ajith Kumar**

QUESTIONS:05 **Total Marks:25**

Analytical Problem Solving

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks with Answers:

a) Identify the conceptual error in the likelihood formulation.

Error:

The likelihood function was written as the sum of probabilities instead of the product of probabilities.

- Incorrect formulation:
  - $L(\theta) = \sum_{i=1}^{n} P(x_i|\theta)$

- Correct formulation:

$$L(\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$

Reason:

In MLE, the observations are assumed to be **independent**, so the probability of observing the entire dataset is obtained by multiplying the probabilities of individual observations.

b) Explain how this mistake affects parameter estimation.

Using the sum instead of the product causes:

- The likelihood no longer represents the probability of the complete dataset.
- The optimization objective becomes mathematically incorrect.
- Estimated parameters do not maximize the true likelihood.
- Even with a large dataset, the model converges to incorrect parameter values.
- This results in poor prediction accuracy and unreliable classification.

c) Suggest the correct mathematical formulation and reasoning.
For a dataset with $n$independent samples:

$$L(\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$

The MLE estimate is obtained by finding the parameter that maximizes the likelihood:

$$\hat{\theta} = argmax_{\theta} L(\theta)$$

Since products are difficult to compute, we usually maximize the **log-likelihood**:

$$l(\theta) = logL(\theta) = \sum_{i=1}^{n} logP(x_i|\theta)$$

The parameter estimate becomes:

$$\hat{\theta} = argmax_{\theta} l(\theta)$$

Reasoning:
Multiplication correctly models the joint probability of independent observations, ensuring statistically valid parameter estimation.

d) How would using log-likelihood fix computational issues?2. Maximum Likelihood Estimation (MLE)
Using the log-likelihood provides several advantages:

- Converts multiplication into addition:

$$log\left(P_i\right) = \sum log(P_i)$$

- Prevents numerical underflow, since multiplying many small probabilities can become nearly zero.
- Simplifies differentiation and optimization.
- Makes gradient-based optimization faster and more stable.
- Produces the same optimal parameter values because the logarithm is a monotonically increasing function.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.
Possible implementation errors include:

- Incorrect weight update formula.
- Bias term is missing or not updated.
- Wrong prediction rule (incorrect threshold function).
- Learning rate is set incorrectly.
- Weights are initialized improperly.
- Features or labels are coded incorrectly (e.g., using 0/1 instead of −1/+1 without adjusting the algorithm).
- Training loop stops too early or update condition is incorrect.

b) Analyze how incorrect learning rate or weight updates affect convergence.

- **Very high learning rate:** Weights change too much, causing the algorithm to overshoot the solution and fail to converge.
- **Very low learning rate:** Learning becomes very slow and may require many iterations.
- **Incorrect weight update:** If the update rule is wrong, the decision boundary moves in the wrong direction, preventing convergence.
    - **Correct Perceptron update rule:**
    - $w = w + \eta(y - \hat{y})x$

where:

- w= weight vector
- η= learning rate
- y= actual label
- $\hat{y}$= predicted label
- x= input feature vector

c) Identify dataset-related issues that may cause this problem.
Possible dataset issues include:

- Data is not actually linearly separable.
- Incorrect or noisy class labels.
- Outliers affecting the decision boundary.
- Features have very different scales (lack of normalization).
- Duplicate or inconsistent training samples.

d) Propose modifications to ensure convergence.3. Building a Machine Learning Algorithm (Model Selection)

To improve convergence:

- Use the correct Perceptron weight update rule.
- Include and update the bias term.
- Choose a suitable learning rate (e.g., 0.01–0.1).
- Normalize or standardize input features.
- Verify that the dataset is linearly separable.
- Correct mis labeled or noisy samples.
- Shuffle training data before each epoch.
- Train until no misclassifications occur or the maximum number of epochs is reached.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.
Possible errors include:

- Incorrect derivative of the activation function.

- Wrong implementation of the chain rule during backpropagation.
- Gradient sign error (adding instead of subtracting gradients).
- Incorrect learning rate.
- Bias gradients not updated.
- Dimension or matrix multiplication errors.
- Weights not updated after each iteration.
- Loss function implemented incorrectly.

b) Explain the role of activation functions in gradient flow.

Activation functions introduce non-linearity, enabling neural networks to learn complex patterns.

Their derivatives determine how gradients flow backward through the network:

- Sigmoid: Can cause very small gradients for large positive or negative inputs.
- Tanh: Zero-centered but still suffers from vanishing gradients.
- ReLU: Reduces vanishing gradients for positive inputs but may produce "dead neurons" when outputs become zero.
- Leaky ReLU: Prevents dead neurons by allowing a small gradient for negative inputs.

c) Analyze how vanishing or exploding gradients affect training.

- **Vanishing Gradients**

- Gradients become extremely small.
- Early layers receive almost no updates.
- Learning becomes very slow or stops.
- Training loss decreases very slowly or remains constant.

- **Exploding Gradients**

- Gradients become extremely large.
- Weight updates become unstable.

- Loss increases rapidly or becomes NaN (Not a Number).
- Training fails to converge.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

To improve training stability:

- Use ReLU or Leaky ReLU activation functions.
- Apply Xavier (Glorot) or He weight initialization.
- Normalize inputs using feature normalization or batch normalization.
- Use an appropriate learning rate (or a learning rate scheduler).
- Apply gradient clipping to prevent exploding gradients.
- Use optimizers such as Adam or RMSProp.
- Verify the correctness of gradient calculations and backpropagation code.

4. **Gradient Descent vs SGD Error Investigation**

A model trained using Gradient Descent converges very
slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.
Batch Gradient Descent computes the gradient using the entire training dataset before updating the model weights.
This makes it slow because:

- Every update requires processing all training samples.
- Computational cost increases with dataset size.
- Requires high memory usage.
- Weight updates occur less frequently, leading to slower convergence.

b) Analyze the cause of fluctuations in SGD.
Stochastic Gradient Descent (SGD) updates the weights using one training sample at a time.

This causes fluctuations because:

- Each sample provides a noisy estimate of the true gradient.
- Weight updates vary significantly between samples.
- Loss may increase or decrease from one iteration to the next.
- Although noisy, these fluctuations can help escape local minima and saddle points.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Mini-Batch Gradient Descent uses a small batch of samples (e.g., 32, 64, or 128) for each update.

Its error behaviour is:

- More stable than SGD because gradients are averaged over multiple samples.
- Faster than Batch Gradient Descent due to smaller computations.
- Less noisy than SGD, resulting in smoother convergence.
- Offers a good balance between speed and stability.

d) Recommend the best approach and justify with error trade-offs.

Recommended Approach: Mini-Batch Gradient Descent

Justification:

- Faster than Batch Gradient Descent.
- More stable than SGD.
- Reduces gradient noise while maintaining efficient updates.
- Uses less memory than full-batch methods.
- Widely used in deep learning because it provides good convergence and computational efficiency.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data
performs well on training data but poorly on test data due to overfitting,
related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.
High dimensionality increases model error because:

- The number of features becomes very large compared to the number of training samples.
- Data becomes sparse, making it difficult to learn meaningful patterns.
- The model learns noise instead of the true relationship (**overfitting**).
- More features increase model complexity, reducing generalization to new data.

b) Identify signs of overfitting in this scenario.

Signs of overfitting include:

- Very high training accuracy but low test accuracy.
- Low training error and high testing error.
- The model performs well on training data but poorly on unseen data.
- Large gap between training and validation performance.

c) Analyze why distance-based learning fails in high dimensions.

Distance-based algorithms (e.g., KNN, K-Means) rely on distances between data points.
In high-dimensional data:

- Distances between points become very similar.
- The difference between nearest and farthest neighbours decreases.
- Meaningful similarity is lost.
- Models cannot accurately identify neighbouring samples, reducing prediction accuracy.

d) Suggest dimensionality reduction or regularization techniques to improve performance.
To improve performance:

- Principal Component Analysis (PCA): Reduces the number of features while preserving most of the data variance.
- Feature Selection: Remove irrelevant or redundant features.
- L1 Regularization (Lasso): Eliminates less important features by shrinking some weights to zero.
- L2 Regularization (Ridge): Reduces model complexity by shrinking weight values.
- Dropout (for Neural Networks): Randomly disables neurons during training to reduce overfitting.
- Increase Training Data: More samples help the model generalize better.